HEAP SORT

Algoritmo de ordenação baseado em ***Heap*** **-** árvore binária especial

Eliane Braga, Enzo Gabriel, Ryan Guedes, Eduardo Pascoal, Mikael Rodrigo, Lucas Henrique

ORIGEM & CONCEITO

Criado por JWJ Williams, em 1964;
O procedimento Build-Max-Heap foi sugerido por Floyd

- É baseado na estrutura de dados conhecida como heap.

Um heap é uma árvore binária especial onde o pai é sempre maior ou igual aos filhos (Max-Heap) ou menor ou igual (Min-Heap)

PASSO A PASSO DO ALGORITMO

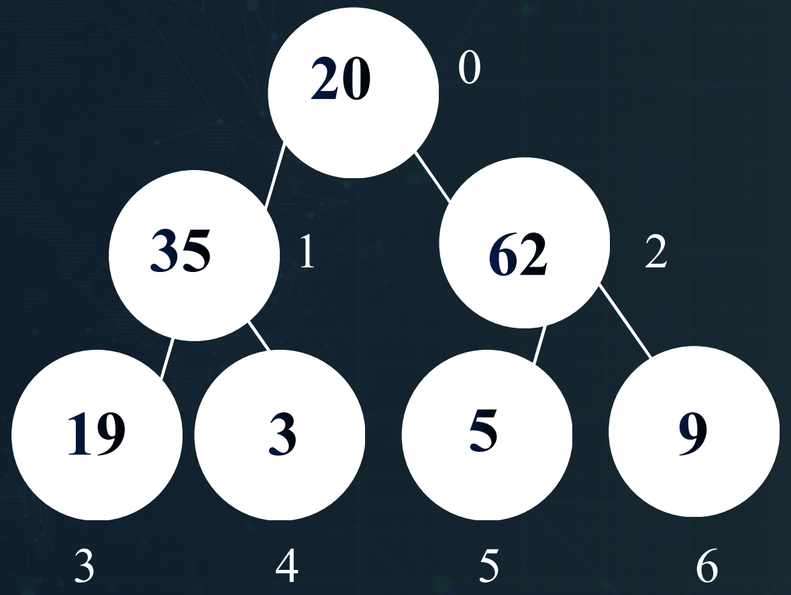
ANTES DA ORDENAÇÃO

Vetor de exemplo:

[20, 35, 62, 19, 3, 5, 9]
0 1 2 3 4 5 6

*Filho esquerdo = 2 * pai + 1*
*Filho direito = 2 * pai + 2*

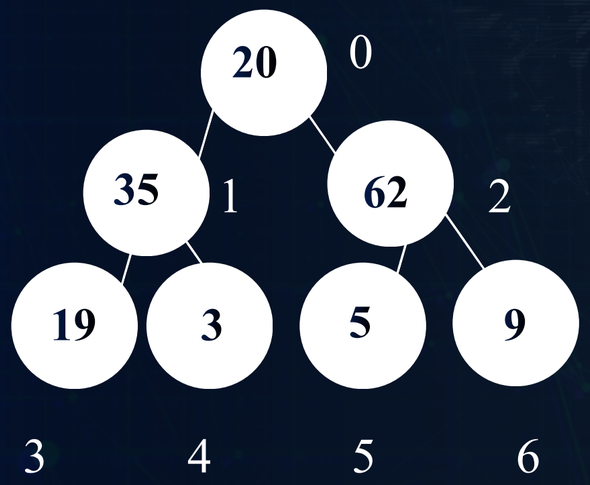
Representação em árvore:



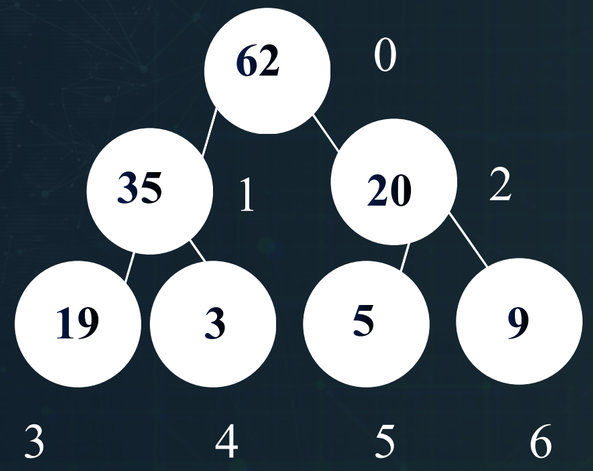
PASSO A PASSO DO ALGORITMO

1. **Construção do Heap Máximo:** A primeira etapa envolve a transformação do array desordenado em um heap máximo

O algoritmo realiza sucessivas comparações, verificando se os filhos são maiores (ou iguais) ao pai



Inverte a posição do 20 pela posição do 62



MAX HEAP

PASSO A PASSO DO ALGORITMO

NO CÓDIGO

```c
void trocar(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Função auxiliar que inverte a posição dos elementos

```c
void max_heapify(int vetor [], int tamanho, int pai){
    int maior = pai;

    int filhoEsquerdo = 2 * pai + 1;
    int filhoDireito = 2 * pai + 2;
```

Função responsável por organizar o Max Heap

PASSO A PASSO DO ALGORITMO

NO CÓDIGO

```
if(filhoEsquerdo < tamanho &&
    vetor[filhoEsquerdo] > vetor[maior]){
    maior = filhoEsquerdo;
}

if(filhoDireito < tamanho &&
    vetor[filhoDireito] > vetor[maior]){
    maior = filhoDireito;
}
```

Verifica se o filho existe e se é maior do que o pai

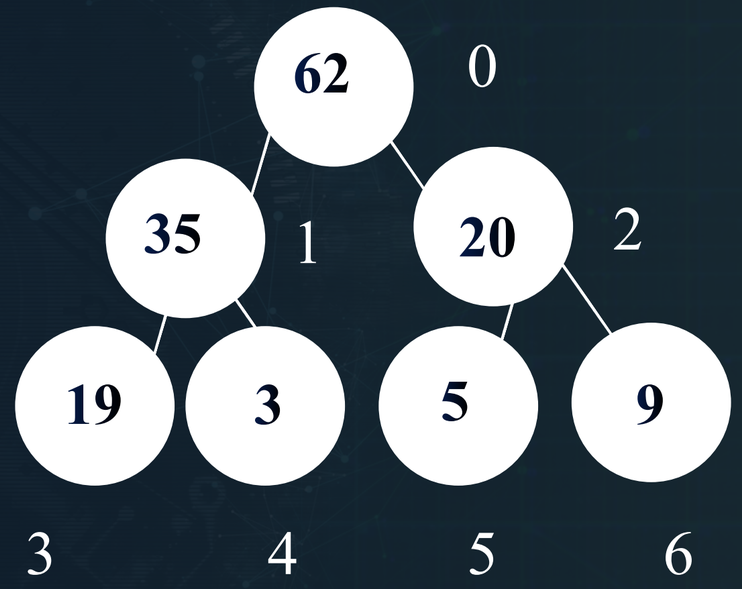
```
if(maior != pai){
    trocar(&vetor[pai], &vetor[maior]);

    max_heapify(vetor, tamanho, maior);
}
```

Se algum dos filhos for maior, assume a posição de pai e a função se chama de forma recursiva para continuar a organização

PASSO A PASSO DO ALGORITMO

2. Ordenação:
Uma vez que o heap máximo tenha sido construído, o maior elemento estará sempre na raiz da árvore (posição 0 do array).

- Esse elemento é trocado com o último elemento do array, que agora está na posição correta
- Em seguida, o heap é reconstruído sem o último elemento, que já está na posição correta.



Após a troca

[9, 35, 20, 19, 3, 5, 62]
0 1 2 3 4 5 6

PASSO A PASSO DO ALGORITMO

NO CÓDIGO

```
void build_heap(int vetor [], int tamanho){
    for(int i = tamanho/ 2 -1; i >= 0; i--){
        max_heapify(vetor,tamanho, i);
    }
}
```

Esta funçao começa do último nó que tem filho e organiza a árvore de baixo para cima, utilizando a função “max _heapify”

PASSO A PASSO DO ALGORITMO

NO CÓDIGO

Esta função organiza o vetor como um heap, o percorre de trás para frente, alterando a posição dos elementos, ao **colocar o maior elemento no final do vetor**

Apenas a parte ainda não ordenada é organizada

```c
void heapSort(int vetor[], int tamanho){
    build_heap(vetor, tamanho);

    for(int i = tamanho - 1; i > 0; i--){
        trocar(&vetor[0], &vetor[i]);

        max_heapify(vetor, i, 0);
    }

}
```

Função Heap Sort

ANÁLISE ASSINTÓTICA

1.

Melhor caso

Vetor já ordenado

O(n log n)

2.

Médio Caso

Vetor com elementos distintos em ordem aleatória

O(n log n)

3.

Pior Caso

Vetor com elementos que exige o máximo de trabalho possível

O(n log n)

O procedimento Heap Sort demora o tempo O(n log n), já que a chamada a Build Max-Heap demora o tempo O(n), e cada uma das n — 1 chamadas a Max-Heapify demora o tempo O(log n).

ESTÁVEL OU ADAPTATIVO?

Nenhum dos dois!!

1. **Por que não é estável?**

   O Heap Sort (a menos que seja implementado de modo especial), não preserva a ordem de elementos iguais, ou seja, não é estável

2. **Por que não é adaptativo?**

   Não é adaptativo, pois independente da ordem prévia de seus elementos, manterá complexidade assintótica O(n log n).

GRAPHS